Pokédex y Recetario en React Native

Guía para documentar en el cuaderno · Método GET · PokeAPI y TheMealDB

1. Descripción de la aplicación

La aplicación está hecha con **React Native** y **Expo**. Tiene dos secciones en el menú inferior: el **Pokédex**, que muestra los primeros **151 Pokémon**, y el **Recetario**, que muestra recetas por categoría. Toda la información se obtiene de APIs públicas usando únicamente el método **GET**. Para hacer las peticiones se usa la librería **axios**. La app se compiló como APK para instalarse en Android.

2. Archivos creados

El proyecto se organiza en tres capas: **api** (conexión), **services** (peticiones y lógica) y **screens** (lo que ve el usuario).

Archivo	Para qué sirve
`api/pokeApi.js`	Configura axios con la dirección base de PokeAPI.
`api/mealApi.js`	Configura axios con la dirección base de TheMealDB.
`services/pokemonService.js`	Funciones que piden los Pokémon a la API.
`services/recetasService.js`	Funciones que piden las recetas y ordenan ingredientes y pasos.
`screens/pokedex/PokedexScreen.js`	Lista de 151 Pokémon con buscador.
`screens/pokedex/PokemonDetalleScreen.js`	Detalle de un Pokémon: tipos, datos y estadísticas.
`screens/pokedex/colores.js`	Colores de cada tipo de Pokémon.
`screens/recetario/RecetarioScreen.js`	Categorías y botones Buscar y Sorpréndeme.
`screens/recetario/CategoriaScreen.js`	Recetas de una categoría.
`screens/recetario/BuscarRecetaScreen.js`	Buscador de recetas por nombre.
`screens/recetario/RecetaDetalleScreen.js`	Detalle de una receta: ingredientes y pasos.
`screens/recetario/ListaRecetas.js`	Componente reutilizable: cuadrícula de recetas.
`App.js`	Navegación: pestañas y pilas de pantallas (lista → detalle).

3. Endpoints utilizados (todos con GET)

PokeAPI — base: https://pokeapi.co/api/v2

Endpoint	Para qué se usa
`/pokemon?limit=151`	Lista de los primeros 151 Pokémon (nombre y url).
`/pokemon/{nombre o id}`	Datos completos de un Pokémon (ej. /pokemon/25).

TheMealDB — base: https://www.themealdb.com/api/json/v1/1

Endpoint	Para qué se usa
`/filter.php?c={categoria}`	Recetas de una categoría (ej. ?c=Seafood).
`/search.php?s={texto}`	Buscar recetas por nombre.
`/lookup.php?i={id}`	Una receta completa por su id.
`/random.php`	Una receta al azar (botón Sorpréndeme).

Imágenes: las de los Pokémon vienen de `raw.githubusercontent.com/PokeAPI/sprites` y las de categorías e ingredientes de `themealdb.com/images`.

4. Estructura de la información que se recibe (modelo)

Solo se muestran los campos que usa la app.

GET /pokemon?limit=151

```
{
  "count": 1351,
  "results": [
    { "name": "bulbasaur", "url": "https://pokeapi.co/api/v2/pokemon/1/" }
  ]
}
```

GET /pokemon/25

```
{
  "id": 25,
  "name": "pikachu",
  "height": 4,            // en decímetros  -> 0.4 m
  "weight": 60,           // en hectogramos -> 6 kg
  "types":     [ { "type": { "name": "electric" } } ],
  "abilities": [ { "ability": { "name": "static" } } ],
  "stats":     [ { "base_stat": 35, "stat": { "name": "hp" } } ]
}
```

GET /filter.php?c=Seafood

```
{
  "meals": [
    { "idMeal": "52959", "strMeal": "Baked salmon...", "strMealThumb": "https://..." }
  ]
}
```

GET /lookup.php?i=52959 (igual en search.php y random.php)

```
{
  "meals": [
    {
      "idMeal": "52959",
      "strMeal": "Baked salmon with fennel & tomatoes",
      "strCategory": "Seafood",
      "strArea": "British",
      "strInstructions": "Heat oven to 180C...",
      "strMealThumb": "https://...",
      "strIngredient1": "Fennel",   "strMeasure1": "2 medium",
      ...hasta strIngredient20 / strMeasure20
    }
  ]
}
```

5. Conexión con la API (axios)

Se crea una instancia de axios con la dirección base. Así, en los servicios solo se escribe el final de la ruta.

```
// api/pokeApi.js
import axios from 'axios';

const pokeApi = axios.create({
  baseURL: 'https://pokeapi.co/api/v2',
  timeout: 10000,
});
export default pokeApi;
```

Explicación: `axios.create` configura la URL base y un tiempo máximo de espera de 10 segundos. `export default` permite importarlo en otros archivos. mealApi.js es igual, con la URL de TheMealDB.

6. Pokédex

6.1 Función asíncrona: obtener los 151 Pokémon

```
export async function getPokemon(nombreOId) {
  const res = await pokeApi.get(`/pokemon/${nombreOId}`);
  return res.data;
}

export async function getPokemons(cantidad = 151) {
  const res = await pokeApi.get(`/pokemon?limit=${cantidad}`);
  const detalles = await Promise.all(
    res.data.results.map((p) => getPokemon(p.name))
  );
  return detalles.map((p) => ({
    id: p.id,
    nombre: p.name,
    tipo: p.types[0].type.name,
  }));
}
```

Explicación: `async` indica que la función hace algo que tarda y `await` espera la respuesta de la API. Primero se pide la lista de 151. Con **map** se crea una petición GET por cada Pokémon y **Promise.all** las hace al mismo tiempo. Al final, otro **map** transforma cada respuesta en un objeto sencillo con id, nombre y tipo.

6.2 Almacenar la información (useState y useEffect)

```
const [pokemons, setPokemons] = useState([]);
const [cargando, setCargando] = useState(true);

useEffect(() => {
  async function cargar() {
    try {
      setPokemons(await getPokemons(151));
    } catch (e) {
      setError('No se pudo cargar la Pokédex.');
    } finally {
      setCargando(false);
    }
  }
  cargar();
}, []);
```

Explicación: useState guarda la lista en la pantalla; al llamar `setPokemons` la pantalla se vuelve a dibujar con los datos. **useEffect** con `[]` se ejecuta una sola vez, al abrir la pantalla. **try/catch** atrapa el error si no hay internet y **finally** quita el indicador de carga.

6.3 Buscador (filter)

```
const texto = busqueda.trim().toLowerCase();
const filtrados = pokemons.filter(
  (p) => p.nombre.includes(texto) || String(p.id) === texto
);
```

Explicación: filter se queda solo con los Pokémon cuyo nombre contiene el texto o cuyo número es igual al escrito. **trim** quita espacios y **toLowerCase** convierte a minúsculas.

6.4 Mostrar la lista y abrir el detalle (FlatList y navigate)

```
<FlatList
  data={filtrados}
  numColumns={2}
  keyExtractor={(p) => String(p.id)}
  renderItem={({ item }) => (
    <Pressable onPress={() =>
      navigation.navigate('PokemonDetalle', { id: item.id })}>
      <Image source={{ uri: imagenPokemon(item.id) }} />
      <Text>{numero(item.id)}</Text>
      <Text>{item.nombre}</Text>
    </Pressable>
  )}
/>
```

Explicación: FlatList recorre el arreglo y dibuja una tarjeta por Pokémon en 2 columnas. **keyExtractor** da un identificador único a cada tarjeta. Al tocarla, **navigate** abre la pantalla de detalle y le manda el id.

6.5 Detalle: recibir el id y mapear estadísticas

```
const { id } = route.params;

useEffect(() => {
  async function cargar() {
    const datos = await getPokemon(id);
    setPokemon(datos);
  }
  cargar();
}, [id]);

{pokemon.stats.map((s) => (
  <View key={s.stat.name}>
    <Text>{nombresStats[s.stat.name]}</Text>
    <Text>{s.base_stat}</Text>
    <View style={{ width: `${(s.base_stat / 255) * 100}%` }} />
  </View>
))}
```

Explicación: route.params recibe el id que mandó la lista. Con ese id se hace el GET del Pokémon. Después, **map** recorre sus estadísticas y por cada una muestra el nombre en español, el valor y una barra cuyo ancho es el porcentaje respecto al máximo (255). La altura y el peso se dividen entre 10 para pasarlos a metros y kilogramos.

7. Recetario

7.1 Funciones asíncronas del servicio

```
export async function getRecetasPorCategoria(categoria) {
  const res = await mealApi.get(`/filter.php?c=${categoria}`);
  return res.data.meals ?? [];
}

export async function getReceta(id) {
  const res = await mealApi.get(`/lookup.php?i=${id}`);
  return res.data.meals[0];
}

export async function getRecetaAleatoria() {
  const res = await mealApi.get('/random.php');
  return res.data.meals[0];
}
```

Explicación: cada función hace un GET a un endpoint distinto. La API siempre responde con un arreglo `meals`. Si no hay resultados manda **null**, por eso se usa `?? []` para regresar un arreglo vacío. Para una sola receta se toma la posición `[0]`. buscarRecetas funciona igual con /search.php?s=

7.2 Ordenar los ingredientes (ciclo for)

```
export function obtenerIngredientes(receta) {
  const ingredientes = [];
  for (let i = 1; i <= 20; i++) {
    const nombre = receta[`strIngredient${i}`];
    const cantidad = receta[`strMeasure${i}`];
    if (nombre && nombre.trim() !== '') {
      ingredientes.push({ nombre, cantidad });
    }
  }
  return ingredientes;
}
```

Explicación: la API no manda una lista de ingredientes, sino 20 campos separados (strIngredient1 a strIngredient20). El ciclo **for** los recorre, ignora los vacíos y con **push** los guarda en un arreglo de objetos con nombre y cantidad, que después se recorre con map.

7.3 Separar la preparación en pasos (split y filter)

```
export function obtenerPasos(receta) {
  return receta.strInstructions
    .split(/\r?\n/)
    .map((linea) => linea.trim())
    .filter((linea) => linea !== '' && !/^step\s*\d+$/i.test(linea));
}
```

Explicación: split corta el texto en cada salto de línea, **map** quita espacios a cada línea y **filter** elimina las líneas vacías y las que solo dicen "Step 1". El resultado es un arreglo de pasos que se muestra numerado.

7.4 Botón Sorpréndeme y paso de datos entre pantallas

```
async function sorprendeme() {
  const receta = await getRecetaAleatoria();
  navigation.navigate('RecetaDetalle', {
    id: receta.idMeal,
    titulo: receta.strMeal,
  });
}
```

Explicación: espera la receta al azar y abre el detalle mandando su id y su nombre. La pantalla de detalle los recibe con `route.params` y hace el GET a /lookup.php con ese id.

7.5 Mostrar ingredientes y pasos (map)

```
{ingredientes.map((ing, i) => (
  <View key={i}>
    <Text>{ing.nombre}</Text>
    <Text>{ing.cantidad}</Text>
  </View>
))}

{pasos.map((paso, i) => (
  <Text key={i}>{i + 1}. {paso}</Text>
))}
```

Explicación: map recorre cada arreglo y crea un elemento por ingrediente y por paso. El índice `i` sirve como clave y para numerar los pasos.

8. Pantallas y navegación

Pantalla	Qué muestra	Endpoint
Pokédex	Cuadrícula de 151 Pokémon y buscador	`/pokemon?limit=151`
Detalle Pokémon	Imagen, tipos, altura, peso, habilidades y estadísticas	`/pokemon/{id}`
Recetario	6 categorías, Buscar y Sorpréndeme	`/random.php`
Categoría	Recetas de la categoría elegida	`/filter.php?c=`
Buscar	Resultados al escribir un nombre	`/search.php?s=`
Detalle receta	Imagen, ingredientes y pasos	`/lookup.php?i=`

En **App.js** se usa **createBottomTabNavigator** para las pestañas del menú y **createNativeStackNavigator** para ir de la lista al detalle con flecha de regresar.

9. Despliegue en Android

- Se agregó **versionCode** en app.json.
- `npx expo prebuild -p android` genera la carpeta android.
- `cd android` y `.\gradlew.bat assembleRelease` compila el APK.
- El APK queda en android/app/build/outputs/apk/release/app-release.apk y se instala en el celular.